

Week 15: XCM & Integration Summary

Planned Deliverables (from thesis plan)

- Implement cross-chain message handlers and parachain transfers
- Integrate Ethereum bridge
- Develop chain extensions for efficiency
- Test initial integrations

What Was Built

1. XCM Cross-Chain Extrinsic

Location: pallets/content-rights/src/lib.rs

Cross-chain operations implemented as pallet extrinsics that construct and send XCM messages:

Table 1: XCM Cross-Chain Extrinsic

Operation	Extrinsic	XCM Flow
Cross-chain subscription renewal	<code>renew_subscription_cross_chain</code>	Source parachain → Target parachain via <code>WithdrawAsset</code> + <code>BuyExecution</code> + <code>Transact</code>
Cross-chain PPV access	<code>pay_per_view_cross_chain</code>	Same pattern as renewal
Cross-chain ownership transfer	<code>transfer_ownership_cross_chain</code>	Source → Target with ownership state update

XCM config changes (runtime/src/configs/xcm_config.rs):

- `RelayNetwork` set to `Some(NetworkId::ByGenesis(ROCOCO_GENESIS_HASH))`
- `UniversalLocation` includes `GlobalConsensus` for absolute location resolution
- `LocationToAccountId` extended with `HashedDescription`, `GlobalConsensusParachainConvertsFor` for Ethereum origins
- Paid execution barriers (`WithdrawAsset` + `BuyExecution`, not `UnpaidExecution`)

Relevant commits:

- 9d88646 — Add XCM cross-chain extrinsics for content rights operations
- 25265ee — Add Snowbridge XCM config (Ethereum origin converters, `GlobalConsensus`)

2. Ethereum Bridge Integration (Snowbridge)

Full local Snowbridge v1 bridge setup connecting Ethereum to the Content Rights parachain:

Infrastructure:

- 4-chain Zombienet topology (Relay, Bridge Hub 1013, AssetHub 1000, Content Rights 100)
- Geth v1.17.1 (Ethereum execution layer)
- Lodestar v1.35.0 (Ethereum consensus/beacon layer, mainnet preset)
- 16 Gateway contracts deployed on Ethereum via Forge
- Beacon light client initialized on Bridge Hub
- Beacon relay + Ethereum relay running

Key scripts:

- scripts/snowbridge-full-setup.sh — Automated end-to-end Ethereum side setup
- scripts/open-hrmp-snowbridge.mjs — HRMP channel setup for 4-chain topology
- scripts/configure-snowbridge.mjs — Substrate-side Snowbridge configuration
- scripts/deploy-gateway.sh — Contract deployment + BEEFY checkpoint

Relevant commits:

- 25265ee — Snowbridge local bridge setup
- 7b967d8 — Automated full setup script

Documentation:

- docs/SNOWBRIDGE_SETUP.md — Setup plan (7 phases)
- docs/SNOWBRIDGE_SESSION_LOG.md — Detailed session log with lessons learned

3. Chain Extensions / Precompiles

Instead of traditional chain extensions, a **custom pallet-revive precompile** was implemented for efficient contract-to-pallet calls:

- Precompile address: 0x0000...0400
- Function: `check_access(content_id, account)` — returns access status
- Allows ink! contracts to query pallet state without cross-contract calls

Relevant commit: 7bc9651

4. Cross-Chain Storage Proof Verification

Location: pallets/rights-verifier/

pallet-rights-verifier enables trustless cross-chain verification of content rights by verifying Merkle storage proofs from remote parachains:

- `verify_rights_proof` — Verifies a storage proof against a relay chain state root
- Enables a parachain to confirm subscription/ownership status on another parachain without XCM round-trips

Relevant commit: 696d990

Deviations from Plan

Table 2: Deviations from Plan

Planned	Actual	Rationale
Chain extensions (substrate)	Custom pallet-revive precompile	pallet-revive uses PolkaVM, not WASM — precompiles are the equivalent of chain extensions
Simple bridge integration	Full Snowbridge v1 local setup	Thesis requires demonstrating heterogeneous blockchain interop; Snowbridge is the production Polkadot ↔ Ethereum bridge
Basic XCM messages	Paid execution XCM with proper fee handling	Realistic XCM requires WithdrawAsset + BuyExecution, not unpaid shortcuts